



Figure 6: Screenshot 1



Figure 7: Screenshot 2

Using tools to make life easy with MicroPython

Adafruit MicroPython Tool (Ampy): This is a utility to interact with a MicroPython board over a serial connection. Ampy is meant to be a simple command line tool to manipulate files and run code on a MicroPython board over its serial connection. With

it, you can send files from your computer to a MicroPython board's file system, download files from a board to your computer, and even send a Python script to a board to be executed. Download Ampy using Python Pip.

```
sudo pip3 install adafruit-ampy
```

Transfer files from PC to board using Ampy and vice versa, as follows:

```
ampy --port "serial_port" put "file_name.py"
ampy --port "serial_port" get "file_name.py"
```

```
iAyan:~ iAyan$ ampy --port /dev/tty.SLAB_USBtoUART get boot.py
# This file is executed on every boot (including wake-boot from deepsleep)
#import esp
#esp.osdebug(None)
import gc
import webrepl
webrepl.start()
gc.collect()
```

```
iAyan:~ iAyan$ ampy --port /dev/tty.SLAB_USBtoUART put main.py
iAyan:~ iAyan$ ampy --port /dev/tty.SLAB_USBtoUART ls
boot.py
webrepl_cfg.py
main.py
```



Figure 8: Screenshot 3

This is handy to transfer `main.py` file to board which will run as board boots after `boot.py` file.

Hardware tinkering with MicroPython

Hardware hacking - GPIO: Connect the positive side, i.e., the anode of the LED to pin no. 15 on the ESP8266 board via a 330-ohm resistor and the negative side or the cathode (the shorter leg) to the ground pin.

```
import machine
import time

pin_number = 15
pin = machine.Pin(pin_number, machine.Pin.OUT)

while True:
    pin.high()
    time.sleep(1)
    pin.low()
    time.sleep(1)
```

Networking with MicroPython

Networking is a powerful feature of microcontrollers today, thanks to the IoT era. ESP8266 is known to be a low cost Wi-Fi development board and, thanks to MicroPython, it exposes the required APIs to access networking on the ESP8266 board.

Here's a script to connect your board to your router and open the gate to the Internet:

```
>>> import network
>>> AP_NAME = " "
>>> PASSWORD = " "
>>> ap = network.WLAN(network.STA_IF)
>>> ap.active(True)
#5 ets_task(4020ed88, 28, 3fff9788, 10)
>>> ap.connect(AP_NAME, PASSWORD)
>>> ap.ifconfig()
('192.168.1.127', '255.255.255.0', '192.168.1.1', '192.168.1.1')
```

Congratulations! ESP8266 is now a device on your local network with just five lines of Python code. Now you can access WebREPL over this IP address. 

By: Ayan Pahwa

The author works as an embedded software engineer at Mentor Graphics (Siemens) and is also technical director and co-founder of SDIOT (Smart Devices for IoT). You can view his recent contributions at www.github.com/ayanpahwa.